Doubly Linked List – Insert After Element –Space Complexity and Algorithm

Program :

```
void insertAfterElement(int data, int element)
{
    Node *temp = head;
    while (temp != nullptr && temp->data != element)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Element " << element << " not found in the
list." << endl;
        return;
    }

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;

    newNode->next = temp->next;
    newNode->prev = temp;
    temp->next = newNode;
    if (newNode->next != nullptr)
    {
        newNode->next->prev = newNode;
    }
    cout << data << " inserted after " << element << "." <<
endl;
}

…….
```

```
        case 4:
            cout << "Enter data to insert: ";
            cin >> data;
            cout << "Enter the element to insert after: ";
            cin >> element;
            insertAfterElement(data, element);
            break;
```

Space Complexity

Auxiliary Space

Auxiliary space is the extra or temporary space used by an algorithm, excluding the space taken up by the inputs.

In the insertAfterElement function, the memory usage is:

*Node *temp: A single temporary pointer is created to find the element. The size of a pointer is constant.*

*Node *newNode: One new node is allocated to hold the new data. The size of this node (sizeof(Node)) is also constant.*

Since the amount of extra memory used does not change with the size of the linked list, the auxiliary space complexity is O(1). The while loop affects the function's time complexity but not its space complexity.

Auxiliary Space: O(1)

Input Space

Input space is the memory used to store the function's inputs. This can be analyzed from two perspectives.

1. Parameter-only Definition

This view considers only the space taken by the function's direct parameters: void insertAfterElement(int data, int element).

The inputs are two integers, data and element, each of which occupies a fixed, constant amount of memory.

Under this common definition, the input space is constant.

Input Space (Parameter-only): O(1)

2. Full-Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list itself.

If the linked list contains n nodes, the total memory it occupies is proportional to n.

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): O(n)

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$\underbrace{\boldsymbol{Space} = \boldsymbol{O(1)}}_{\textit{Auxiliary}} + \underbrace{\boldsymbol{O(1)}}_{\textit{Input}} = \boldsymbol{O(1)}.$$

- **Holistic Interpretation (Full-data input):**

$$\underbrace{\boldsymbol{Space} = \boldsymbol{O(1)}}_{\textit{Auxiliary}} + \underbrace{\boldsymbol{O(n)}}_{\textit{Input}} = \boldsymbol{O(n)}.$$

In most academic and interview contexts, when asked for the "space complexity" of a function, the expected answer is the auxiliary space complexity, which is O(1).

Algorithm: Insert After Element (Doubly Linked List)

$DINSAFTER(START, ITEM, ELEMENT)$
$\left((Node*)malloc\left(sizeof(Node)\right)\ is\ availability\ of\ space\ for\ insertion\ of\ Data,\right.$
$\left.hence\ ,lets\ name\ it\ AVAIL.\right)$

1. [$TRAVERSE\ LIST\ TO\ FIND\ ELEMENT$]

   $Set\ PTR := START$ [$PTR\ is\ a\ pointer\ initialized\ to\ the\ start\ of\ the\ list$].

   $Repeat\ while\ PTR \neq NULL\ AND\ INFO[PTR] \neq ELEMENT$:

   $Set\ PTR := NEXTLINK[PTR]$ [$Updates\ PTR\ to\ the\ next\ node$].

   [$END\ of\ Loop$]

2. [$CHECK\ IF\ ELEMENT\ WAS\ FOUND$]

   $If\ PTR = NULL, then$:

   $Write$: "$Element\ not\ found\ in\ the\ list.$" $and\ EXIT$.

   [$End\ of\ If\ structure$.]

3. [$ALLOCATE\ NEW\ NODE$]

   $Set\ NEW := AVAIL.$

4. [$COPY\ DATA\ INTO\ NEW\ NODE$]

   $Set\ INFO[NEW] := ITEM.$

5. [$LINK\ THE\ NEW\ NODE\ INTO\ THE\ LIST$]

   $a.Set\ NEXTLINK[NEW] := NEXTLINK[PTR]$ [$New\ node's\ NEXT\ points\ to\ what\ was\ after\ PTR$].

   $b.Set\ PREVLINK[NEW] := PTR$ [$New\ node's\ PREV\ points\ back\ to\ PTR$].

   $c.Set\ NEXTLINK[PTR] := NEW$ [$PTR\ now\ points\ forward\ to\ the\ NEW\ node$].

   $d.If\ NEXTLINK[NEW] \neq NULL, then$: [$This\ handles\ cases\ where\ the\ node\ is\ not\ inserted\ at\ the\ very\ end$].

   $Set\ PREVLINK[NEXTLINK[NEW]] := NEW\ \begin{pmatrix}The\ node\ that\ was\ originally\ after\ PTR\ must\\ now\ point\ its\ PREV\ back\ to\ the\ NEW\ node\end{pmatrix}.$

   [$End\ of\ If\ structure$]

6. [$OUTPUT\ THE\ INSERTED\ ITEM\ AFTER\ ELEMENT$]

   $Write: ITEM$, "$inserted\ after$", $ELEMENT$.

7. $Exit$